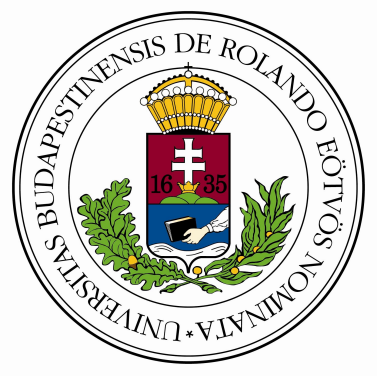
Konkurens programozás

wait/notify



Kozsik Tamás

ELTE Eötvös Loránd Tudományegyetem

Hogyan implementáljunk egy `BlockingQueue`-t?

- Kiindulás: pl. egy `java.util.List`
- Becsomagolva, csak sorműveletek
- Monitorral védett műveletek: szálbiztos
- Üres/megtelt adatszerkezet kezelése: blokkolás



Hogyan implementáljunk egy BlockingQueue-t?

- Kiindulás: pl. egy java.util.List
- Becsomagolva, csak sorműveletek
- Monitorral védett műveletek: szálbiztos
- Üres/megtelt adatszerkezet kezelése: blokkolás

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();
    private final int capacity;
    public Buffer( int capacity ){ this.capacity = capacity; }
    public synchronized void put( T item ){ ... }
    public synchronized T take(){ ... }
}
```



Korlátlan buffer: busy waiting

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
    }

    public synchronized T take(){
        while( impl.isEmpty() ){ /* do nothing */ }
        return impl.removeFirst();
    }
}
```



Korlátlan buffer: busy waiting

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
    }

    public synchronized T take(){
        while( impl.isEmpty() ){ /* do nothing */ }
        return impl.removeFirst();
    }
}
```

a synchronized és a busy waiting itt holtpontot eredményez



Holtpont elkerülése: szétbontott synchronized?

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
    }

    public T take(){
        while( true ){
            synchronized(this){ if( !impl.isEmpty() ) break; }
        }
        synchronized(this){ return impl.removeFirst(); }
    }
}
```



Holtpont elkerülése: szétbontott synchronized?

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
    }

    public T take(){
        while( true ){
            synchronized(this){ if( !impl.isEmpty() ) break; }
        }
        synchronized(this){ return impl.removeFirst(); }
    }
}
```



Check-then-act: nem szétbontható!

Holtpont elkerülése: szétbontott synchronized

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
    }

    public T take(){
        while( true ){
            synchronized(this){
                if( !impl.isEmpty() ) return impl.removeFirst();
            }
        }
    }
}
```



Busy waiting elkerülése?

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
    }

    public T take(){
        while( true ){
            synchronized(this){
                if( !impl.isEmpty() ) return impl.removeFirst();
            }
            Thread.yield();  // vigyázat: nem ad garanciát!
        }
    }
}
```



`wait()` és `notify()`

- Feltételre várakoztatás (blokkoló művelet!)
- Egy szál vár, amíg egy másik szól neki, hogy mehet
- A `java.lang.Object` műveletei
- Kezdetektől fogva
- Van jobb megoldás is a `BlockingQueue`-hoz...



Korlátlan buffer

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
        notify();
    }

    public synchronized T take() throws InterruptedException {
        while( impl.isEmpty() ){
            wait();
        }
        return impl.removeFirst();
    }
}
```



Hamis ébredés

- Ok nélkül is felébredhet a szál a `wait`-ből
- Spurious wake

```
while( impl.isEmpty() ){
    wait();
}
```



Kell a synchronized ahhoz, amire hívjuk

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();

    public synchronized void put( T item ){
        impl.addLast(item);
        this.notify();
    }

    public synchronized T take() throws InterruptedException {
        while( impl.isEmpty() ){
            this.wait();
        }
        return impl.removeFirst();
    }
}
```



Általában

```
synchronized(obj){
    ...
    obj.notify();
    ...
}
```

```
synchronized(obj){
    ...
    while( ... ) obj.wait();
    ...
}
```



Működés

wait-set

- az *A* szál megszerzi `obj` kulcsát (sorbanállás után)
- `obj.wait()` elengedi a kulcsot, a szál bekerül a *wait-set*be
- a *B* szál megszerzi `obj` kulcsát (sorbanállás után)
- `obj.notify()` felébreszt egy alvót a *wait-set*ből
- a felébredt *C* szál sorbanáll a kulcsért
- a *B* szál a `synchronized` végén elengedi a kulcsot
- a *C* szál vár a kulcsra
- ha a *C* szál megszerzi a kulcsot, visszatér az `obj.wait()`-ből
- a `synchronized` végén a *C* elengedi `obj` kulcsát



Megszakítás

- a várakozó szálaknak kívülről jelzünk: “megszakítjuk”
- vannak műveletek, amelyek erre fel vannak készítve
  - a `wait` befejeződhet `InterruptedException` kivétellel
  - a `Thread.sleep(int)` is ilyen
- ellenőrzött kivétel (checked exception)

```
public synchronized T take() throws InterruptedException {
    while( impl.isEmpty() ){
        this.wait();
    }
    return impl.removeFirst();
}
```



Időkorlátos várakozás

- Ha egy komponens lehal, ne rántsa maga után a rendszert
- Robusztusság
- Egy szál ne várjon potenciálisan végtelen ideig másokra



Időkorlát beépítése blokkoló műveletbe

```
import java.util.concurrent.TimeoutException;
...
public class Buffer<T> {
...
   public synchronized T poll( int timeout ) throws InterruptedException,
                                                    TimeoutException {
      long deadline = System.currentTimeMillis() + timeout;
      while( impl.isEmpty() ){
         long current = System.currentTimeMillis();
         if( current >= deadline ){
            throw new TimeoutException();
         } else {
            this.wait( deadline - current );
         }
      }
      return impl.removeFirst();
   }
}
```



Korlátos buffer

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {

   private final List<T> impl = new LinkedList<T>();
   private final int capacity;

   public Buffer( int capacity ){
      if( capacity <= 0 ){
         throw new IllegalArgumentException();
      }
      this.capacity = capacity;
   }

   public synchronized void put( T item ) throws InterruptedException ...
   public synchronized T take() throws InterruptedException ...
}
```



Korlátos buffer egy termelő és egy fogyasztó esetén

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
   private final List<T> impl = new LinkedList<T>();
   private final int capacity;
   public Buffer( int capacity ){ ... }

   public synchronized void put( T item ) throws InterruptedException {
      while( impl.size() == capacity ){ this.wait(); }
      impl.addLast(item);
      this.notify();
   }

   public synchronized T take() throws InterruptedException {
      while( impl.isEmpty() ){ this.wait(); }
      this.notify();
      return impl.removeFirst();
   }
}
```



Több termelő és több fogyasztó esetén?

- Nem tudjuk szabályozni, hogy a `notify()` melyiket ébressze fel
- Ugyanazt a kulcsot és *wait-set*et kell használni
- Mindenkit ébresszünk fel: `notifyAll()`
- Thundering herd problem



Korlátos buffer

```
import java.util.List, java.util.LinkedList;
public class Buffer<T> {
    private final List<T> impl = new LinkedList<T>();
    private final int capacity;
    public Buffer( int capacity ){ ... }

    public synchronized void put( T item ) throws InterruptedException {
        while( impl.size() == capacity ){ this.wait(); }
        impl.addLast(item);
        this.notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while( impl.isEmpty() ){ this.wait(); }
        this.notifyAll();
        return impl.removeFirst();
    }
}
```



Explicit várakoztató: Condition

```
package java.util.concurrent.locks;
public interface Condition {
    void signal();
    void signalAll();
    void await() throws InterruptedException;
```



Explicit várakoztató: Condition

```
package java.util.concurrent.locks;
public interface Condition {
    void signal();
    void signalAll();
    void await() throws InterruptedException;

    boolean await( long time, TimeUnit unit )
                                        throws InterruptedException
    long awaitNanos( long nanosTimeout )
                                        throws InterruptedException
    boolean awaitUntil( Date deadline )
                                        throws InterruptedException
    void awaitUninterruptibly();
}
```



Korlátos buffer (1)

```
import java.util.List, java.util.LinkedList;
import java.util.concurrent.locks.*;
public class Buffer<T> {

    private final Lock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    private final List<T> impl = new LinkedList<T>();
    private final int capacity;

    public Buffer( int capacity ){
        if( capacity <= 0 ) throw new IllegalArgumentException();
        this.capacity = capacity;
    }

    public void put( T item ) throws InterruptedException { ... }
    public T take() throws InterruptedException { ... }
}
```



Korlátos buffer (2)

```
    public void put( T item ) throws InterruptedException {
        lock.lock();
        try {
            while( impl.size() == capacity ){ notFull.await(); }
            impl.addLast(item);
            notEmpty.signal();
        } finally { lock.unlock(); }
    }

    public T take() throws InterruptedException {
        lock.lock();
        try {
            while( impl.isEmpty() ){ notEmpty.await(); }
            notFull.signal();
            return impl.removeFirst();
        } finally { lock.unlock(); }
    }
}
```



Explicit zároló

```
package java.util.concurrent.locks;
public interface Lock {
    ...
    void lock();          // kritikus szakasz eleje
    void unlock();        // kritikus szakasz vége
}
```

A beépített (intrinsic) zároláshoz, a synchronizedhoz képest

- Többféle művelet
- Többféle lehetőség
- Kényelmetlenebb használat



Tipikus használat

Előkészítés

```
Lock l = new ReentrantLock();
```

Kritikus szakaszok szálakban

```
l.lock();
try {
    // access the resource protected by this lock
} finally {
    l.unlock();
}
```



Példa

```
import java.util.concurrent.locks.*;
public class Account {

    private final Lock lock = new ReentrantLock();

    private int balance;

    public int balance(){
        lock.lock();
        try { return balance; }
        finally { lock.unlock(); }
    }

    ...
}
```

